

# Numerical Analysis (CSE 0541-2205)

## Concept Questions & Answers — Study Notes

### 1. What is Numerical Analysis? Applications & Limitations

Numerical Analysis is the branch of mathematics that develops and studies algorithms which use numerical approximation to find solutions to mathematical problems that cannot be solved exactly (analytically), or that are too complex to solve by hand. It converts a mathematical problem — such as a non-linear equation, a system of linear equations, an integral, or a differential equation — into a sequence of arithmetic operations that a computer can perform repeatedly to reach an approximate answer within an acceptable error tolerance.

#### *Applications*

- Solving non-linear equations (e.g., Bisection, Newton-Raphson) in engineering design.
- Solving large systems of linear equations (e.g., Gauss elimination, LU-factorization) in structural and electrical network analysis.
- Interpolation and curve fitting for experimental/tabulated data (e.g., sensor readings, census data).
- Numerical integration and differentiation where a closed-form antiderivative does not exist (e.g., area under a curve, rate of change from discrete data).
- Solving ordinary differential equations (Euler's method, Modified Euler's method) in physics and control systems.
- Used extensively in computer graphics, weather forecasting, financial modeling, and scientific simulations.

#### *Limitations*

- Results are approximate, not exact — every method carries some truncation and round-off error.
- Accuracy depends heavily on step size ( $h$ ) and number of iterations; smaller  $h$  generally means more accuracy but more computation.
- Some methods (e.g., Fixed Point Iteration, Newton-Raphson) may diverge if the initial guess or the function does not satisfy convergence conditions.
- Round-off error accumulates with more computations, and very small step sizes can sometimes increase this error.
- Methods can be computationally expensive for large or highly non-linear systems.

### 2. All Root-Finding Methods — Algorithms

#### *(a) Bisection Method*

- Define  $f(x)$ . Choose  $a, b$  such that  $f(a)$  and  $f(b)$  have opposite signs.
- Compute  $c = (a + b) / 2$ .
- If  $f(c) < 0$ , set  $a = c$ ; otherwise set  $b = c$ .
- Repeat until the desired accuracy (tolerance) is reached.

#### *(b) False Position (Regula Falsi) Method*

- Choose a, b such that  $f(a)$  and  $f(b)$  have opposite signs.
- Compute  $c = [a \cdot f(b) - b \cdot f(a)] / [f(b) - f(a)]$  (a weighted point instead of the midpoint).
- If  $f(c) < 0$ , set  $a = c$ ; otherwise set  $b = c$ .
- Repeat until convergence.

#### (c) *Newton–Raphson Method*

- Choose an initial guess  $x_0$  close to the root.
- Find  $f'(x)$  and iterate:  $x_{(n+1)} = x_{(n)} - f(x_{(n)}) / f'(x_{(n)})$ .
- Repeat until  $|x_{(n+1)} - x_{(n)}|$  is smaller than the tolerance.

#### (d) *Fixed Point Iteration Method*

- Rewrite  $f(x) = 0$  in the form  $x = g(x)$ .
- Choose an initial guess  $x_0$  such that  $|g'(x_0)| < 1$  (convergence condition).
- Iterate:  $x_{(n+1)} = g(x_{(n)})$ .
- Stop when the difference between successive iterates is within tolerance.

#### (e) *Bairstow's Method*

- Used to find complex/real roots of polynomials in pairs by dividing the polynomial by a quadratic factor  $(x^2 - rx - s)$ .
- Adjusts  $r$  and  $s$  iteratively (using partial derivatives of the remainder terms) until the remainder becomes negligible.
- Effectively reduces a higher-degree polynomial step by step into quadratic factors, from which roots (real or complex) are found directly.

### 3. When Does the Answer Converge in a Root-Finding Method?

A root-finding method is said to **converge** when successive approximations ( $x_0, x_1, x_2, \dots$ ) get progressively closer to the true root, so that the difference between consecutive iterates (or the value of  $|f(x)|$ ) keeps shrinking toward zero.

- Bisection / False Position: Always converges if the initial interval  $[a, b]$  satisfies  $f(a) \cdot f(b) < 0$ , because the interval containing the root keeps shrinking every iteration.
- Newton-Raphson: Converges (quadratically — very fast) if the initial guess is sufficiently close to the root and  $f'(x)$  is not zero or very small near the root.
- Fixed Point Iteration: Converges only if  $|g'(x)| < 1$  in the neighborhood of the root; the closer  $|g'(x)|$  is to 0, the faster the convergence.
- In practice, convergence is confirmed numerically when  $|x_{(n+1)} - x_{(n)}|$  (or  $|f(x_{(n)})|$ ) becomes smaller than a pre-set tolerance value.

### 4. Absolute Error vs Relative Error

**Absolute Error** is the magnitude of the difference between the true (exact) value and the approximate value:

$$\text{Absolute Error} = | \text{True Value} - \text{Approximate Value} |$$

**Relative Error** expresses the absolute error as a fraction (or percentage) of the true value, giving a sense of the error's significance relative to the size of the quantity being measured:

$$\text{Relative Error} = | \text{True Value} - \text{Approximate Value} | / | \text{True Value} |$$

- Absolute error has the same unit as the measured quantity; relative error is unitless (often given as a percentage).
- Absolute error alone can be misleading — an error of 1 unit is significant for a value of 10 but insignificant for a value of 10,000. Relative error corrects for this by normalizing against the true value.
- In iterative numerical methods, relative error is often preferred as a stopping criterion because it accounts for the scale of the answer.

## 5. What is a Fixed Point / Fixed Point Method? When Does It Diverge?

A **fixed point** of a function  $g(x)$  is a value  $x = p$  such that  $g(p) = p$  — that is, the input and output of the function are the same. The **Fixed Point Iteration Method** finds a root of  $f(x) = 0$  by first rewriting it in the equivalent form  $x = g(x)$ , then generating a sequence of approximations  $x_{(n+1)} = g(x_{(n)})$  starting from an initial guess  $x_0$ , which (under the right conditions) converges to the fixed point  $p$ , which is also the root of  $f(x) = 0$ .

### When It Diverges

- The method diverges (moves away from the root instead of toward it) when  $|g'(x)| \geq 1$  in the neighborhood of the true root.
- A poor choice of the rearrangement  $g(x)$  — even though algebraically equivalent to  $f(x) = 0$  — can make  $|g'(x)|$  large and cause divergence, even if another rearrangement of the same equation would converge.
- Starting with an initial guess  $x_0$  too far from the actual root can also lead to divergence, since the local convergence condition  $|g'(x_0)| < 1$  may not hold there.
- This is why, for the same equation, choosing the right form of  $g(x)$  is critical to guarantee convergence.

## 6. When Does the Answer Not Converge (Diverge)?

In general, a numerical method fails to converge (diverges, oscillates, or converges to a wrong value) when the mathematical conditions required for convergence are violated. Common causes across methods include:

- Bisection/False Position: The initial interval does not bracket a root ( $f(a)$  and  $f(b)$  have the same sign), or the function has multiple/even-order roots in the interval so the sign does not change.
- Newton-Raphson: The derivative  $f'(x)$  is zero or very close to zero near the guess (causing a huge or undefined jump), the initial guess is far from the root, or the function has an inflection point that sends iterates away from the root.
- Fixed Point Iteration:  $|g'(x)| \geq 1$  near the root — the iterations then move farther away from the root instead of closer, oscillating or growing without bound.
- General causes: an unsuitable step size, an incorrect rearrangement of the equation, or a starting guess that lies outside the interval of convergence.

## 7. Numerical Differentiation & Integration — Definitions

**Numerical Differentiation** is the process of estimating the derivative (rate of change) of a function at a point using discrete data values, instead of using the analytical rules of calculus. It typically uses finite-difference formulas (e.g., the Central Difference formula  $f'(x) \approx [f(x+h) - f(x-h)] / 2h$ ) built from function values at nearby points spaced by a step size  $h$ .

**Numerical Integration** (also called quadrature) is the process of estimating the value of a definite integral — the area under a curve between two limits — using a finite number of function evaluations, rather than finding an exact antiderivative. Methods such as the Trapezoidal rule, Simpson's 1/3 rule, and Simpson's 3/8 rule approximate the curve between subdivision points with simple shapes (straight lines or polynomials) whose areas are easy to compute exactly.

- Both techniques are essential when the function is only known at discrete/tabulated points (e.g., experimental data), or when the function is too complex to differentiate/integrate analytically.
- Accuracy of both improves as the step size  $h$  decreases, but excessively small  $h$  can increase round-off error.

## 8. Trapezoidal, Simpson's 1/3, Simpson's 3/8 Rules — Definitions

### *Trapezoidal Rule*

Approximates the area under the curve between each pair of adjacent points by a trapezoid (i.e., the curve is approximated by a straight line between points). The interval  $[a, b]$  is divided into  $n$  equal subintervals ( $n$  can be any value):

$$I = (h/2) \cdot [ (y_0 + y_n) + 2(y_1 + y_2 + \dots + y_{n-1}) ]$$

### *Simpson's 1/3 Rule*

Approximates the curve between every pair of subintervals with a second-degree (quadratic/parabolic) polynomial. It requires  $n$  to be even:

$$I = (h/3) \cdot [ (y_0 + y_n) + 4(y_1 + y_3 + \dots + y_{n-1}) + 2(y_2 + y_4 + \dots + y_{n-2}) ]$$

### *Simpson's 3/8 Rule*

Approximates the curve over every three subintervals using a third-degree (cubic) polynomial. It requires  $n$  to be a multiple of 3:

$$I = (3h/8) \cdot [ (y_0 + y_n) + 3(y_1 + y_2 + y_4 + y_5 + \dots) + 2(y_3 + y_6 + \dots) ]$$

In general, Simpson's rules give higher accuracy than the Trapezoidal rule for the same number of subintervals, because fitting curved (polynomial) segments captures the shape of a smooth function more closely than straight-line segments.

## 9. Tolerance & Step Size — Significance

### *Tolerance*

Tolerance is the maximum acceptable error (or the smallest acceptable difference between successive approximations) that determines when an iterative numerical method should stop. It acts as the stopping criterion for methods like Bisection, Newton-Raphson, and Fixed Point Iteration.

- A tighter (smaller) tolerance gives a more accurate answer but requires more iterations and computation time.
- A looser (larger) tolerance gives a faster but less precise result.
- Choosing tolerance appropriately balances accuracy against computational cost.

### Step Size ( $h$ )

Step size is the fixed spacing between consecutive points ( $x$ -values) used in numerical differentiation, integration, or in dividing an interval  $[a, b]$  into subintervals:  $h = (b - a) / n$ .

- A smaller step size generally reduces truncation error and gives a result closer to the true value.
- However, an extremely small step size can increase round-off error, since more arithmetic operations are performed and small differences between nearly equal numbers get amplified.
- Step size also affects the required form of  $n$  for methods like Simpson's 1/3 rule ( $n$  even) and Simpson's 3/8 rule ( $n$  a multiple of 3), so its choice must respect these constraints.

## 10. Limitations of Each Root-Finding Algorithm

| Method                | Key Limitations                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Bisection             | Slow (linear convergence); requires $f(a) \cdot f(b) < 0$ ; cannot detect multiple/even-order roots within the interval                 |
| False Position        | Can converge slowly when one endpoint stays fixed for many iterations (one-sided convergence); still requires $f(a) \cdot f(b) < 0$     |
| Newton-Raphson        | Requires computing the derivative $f'(x)$ ; fails or diverges if $f'(x) = 0$ or is very small near the root; sensitive to initial guess |
| Fixed Point Iteration | Converges only if $ g'(x)  < 1$ near the root; convergence and its speed strongly depend on the chosen $g(x)$                           |
| Bairstow's Method     | More complex to implement (needs partial derivatives / synthetic division steps); convergence of $(r, s)$ pairs                         |

## 11. Romberg Integration — Definition

Romberg Integration is a numerical technique that improves the accuracy of the Trapezoidal rule by combining Trapezoidal-rule estimates computed at different (successively halved) step sizes, using Richardson's extrapolation, to cancel out the leading error terms.

It builds a table of increasingly accurate integral estimates: the first column contains Trapezoidal-rule results for  $h, h/2, h/4, \dots$ , and each subsequent column applies Richardson's extrapolation to the previous column's values, producing estimates with progressively higher order of accuracy — often reaching very high precision after just a few extrapolation steps.

General Richardson extrapolation relation (as used to build the Romberg table):

$$I(\text{improved}) = I(h/2) + [I(h/2) - I(h)] / (2^k - 1)$$

where  $k$  depends on the extrapolation level. Romberg Integration is valued because it reaches high accuracy without requiring very small (and computationally expensive) step sizes directly.